Web API Recommendation via Exploring Textual and Structural Semantics With Contrastive Learning and Joint Training

Guosheng Kang, *Member, IEEE*, Hongshuai Ren, Wanjun Chen, Jianxun Liu, Buqing Cao, and Yu Xu, *Member, IEEE*

***Abstract*—With the advancement of service computing technology, software developers tend to consume a variety of Web APIs (Application Programming Interfaces, also named Web services) from Web API markets to create feature-rich Mashup applications to save time and cost. Under such a background, the ever-increasing number of Web APIs makes the service discovery become a challenge. Thus, Web API recommendation becomes an effective means for service discovery. However, the existing approaches to Web API recommendation still have limitations in extracting rich semantics sufficiently from functional description documents and service networks, resulting in a limited recommendation performance. To further improve the recommendation performance, this paper proposes an effective Web API recommendation approach via exploring textual and structural semantics with contrastive learning and joint training, named CLJT. On one side, discriminative feature representations from textual and structural semantics could be derived by contrastive learning with information correlation across views. On the other side, the derived representations could be applicable to Web API recommendation by joint training of the representation tasks and the recommendation task. Extensive experiments are conducted over a real-world dataset crawled from ProgrammableWeb.com. The experimental results demonstrate the superiority of the proposed approach compared to the baseline methods.**

***Index Terms*—Mashup creation, Web API recommendation, textual semantic, structural semantic, contrastive learning, joint training.**

Received 27 May 2024; revised 26 August 2024; accepted 8 December 2024. Date of publication 11 December 2024; date of current version 22 April 2025. This work is supported by Educational Commission of Hunan Province of China under Grant No. 23A0359, Foundation of Yunnan Key Laboratory of Service Computing No. YNSC24126, and Postgraduate Scientific Research Innovation Project of Hunan Province under Grant No. CX20240875. The associate editor coordinating the review of this article and approving it for publication was T. Inoue. *(Corresponding author: Jianxun Liu.)*

Guosheng Kang is with the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan 411199, China, and also with the Yunnan Key Laboratory of Service Computing, Yunnan University of Finance and Economics, Kunming 650503, China (e-mail: guoshengkang@gmail.com).

Hongshuai Ren, Wanjun Chen, Jianxun Liu, and Buqing Cao are with the Hunan Provincial Key Laboratory for Services Computing and Novel Software Technology and the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan 411199, China (e-mail: hongshuairen01@gmail.com; cwisdomjer@gmail.com; ljx529@gmail.com; buqingcao@gmail.com).

Yu Xu is with the Cambridge Baker Systems Genomics Initiative and the British Heart Foundation Cardiovascular Epidemiology Unit, Department of Public Health and Primary Care, University of Cambridge, CB2 1TN Cambridge, U.K. (e-mail: yx322@medschl.cam.ac.uk).

Digital Object Identifier 10.1109/TNSM.2024.3515103

I. Introduction

With the advancement of service computing technology, some Web API (Application Programming Interfaces, also named Web services) markets, such as ProgrammableWeb[1] and RapidAPI,[2] have emerged, on which services can be registered and invoked. As a result, the quantity and diversity of Web services appearing in the form of Web APIs have rapidly increased. Web APIs can be combined quickly and easily to form value-added applications, i.e., Mashups, reducing redundant work in the development process and improving development efficiency. Due to the advantages of Web APIs, users or software developers tend to consume Web APIs to solve their daily business or development requirements [1]. With the explosive growth of Web APIs, consumers find it very difficult and time-consuming to select the desirable services that satisfy their functionality requirements. Therefore, Web API recommendation becomes an effective means for service discovery or selection.

In recent years, Web API recommendation has attracted great attention from academia and industry, and a variety of Web API recommendation methods have been proposed. By reviewing these works, they can be broadly divided into three categories: *Content-based Approaches*, *Graph-based Approaches*, and *Hybrid Approaches*. Content-based approaches focus on matching the textual semantics between the candidate service descriptions and the requirement description. Graph-based approaches aim to extract service functionality semantics from service networks, in which some network representation methods are used, like GNN [2], GCN [3], Node2vec [4], etc. To integrate both the textual semantics and structural semantics, hybrid approaches mine the service functionality semantics from both service description documents and service networks. Even though hybrid approaches show great improvement for the Web API recommendation task compared with content-based and graph-based approaches, there is still a drawback: rich semantics cannot be extracted sufficiently from functional description documents and service networks simultaneously, resulting in limited recommendation performance. The main cause is that the representation processes of functional description documents and service networks are conducted independently.

[1]https://www.mulesoft.com/programmableweb
[2]https://rapidapi.com/hub

1932-4537 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

Thus, the textual and structural semantics cannot be fully fused and correlated when mining service functionality semantics.

To further improve the recommendation performance, this paper proposes an effective Web API recommendation approach via exploring textual and structural semantics with **C**ontrastive **L**earning and **J**oint **T**raining, named **CLJT**. Specifically, discriminative feature representations from textual and structural semantics could be derived by contrastive learning with information correlation across views. Meanwhile, the derived representations could be more applicable to Web API recommendation by joint training of the representation tasks and the recommendation task. The main contributions of our work are summarized as follows.

- The textual representation and structural representation can be complemented and correlated with each other via contrastive learning to improve their representations.
- The textual and structural representation tasks and the Web API recommendation task are trained jointly so that the Web API recommendation performance is improved.
- Extensive experiments are conducted over a real-world dataset, and the experimental results demonstrate the proposed approach outperforms the baseline methods.

The rest of the paper is structured as follows. Section II introduces the related work. The framework of the proposed approach is presented in Section III. Subsequently, Section IV introduces the details of the proposed approach. Section V evaluates the proposed Web API recommendation approach through experiments. Finally, Section VI concludes our work.

II. Related Work

Web API recommendation plays a crucial role in the development of service-based systems. Recently, there has been growing attention towards Web API recommendation. The existing works on Web API recommendation could be roughly categorized into three types: *Content-based Approaches*, *Graph-based Approaches*, and *Hybrid Approaches*.

1) Content-based Approaches. Content-based approaches focus on mining the textual semantics from functional description documents and the requirement description texts from users. The main idea is to match the textual semantics between the service descriptions and the requirement description, and then the Top-K Web APIs with the highest matching degree are recommended to the active user [5], [6]. To calculate the matching degree, usually, the textual descriptions are converted into vectors by some topic modeling techniques based on traditional machine learning or advanced deep learning, like TF-IDF [7], LDA [8], LSTM [9], Word2vec [10], Doc2vec [11], BERT [12], etc. Due to the sparsity of textual descriptions, some auxiliary information, like profile [13], interface [14], context [15] [16], tag [17], etc., could be taken into account to supplement the functionality semantics of services. These techniques are also widely used in service classification or clustering [18], [19]. However, the recommendation performance is still limited by only exploring textual descriptions.

2) Graph-based Approaches. Graph-based approaches extract service functionality semantics from service networks derived from the relationships (like composition relation and tag sharing relation) between services [20], [21], [22], [23]. The idea is that service functionality semantics are implicit in the service network structures. In this kind of works, usually some network representation methods are used, like GNN [2], GCN [3], GAT [24], Node2vec [4], LightGCN [23], etc. Specifically, Jiang et al. [25] employ Node2vec to obtain network representation of the Mashup-API interaction network. Meanwhile, Li et al. [26] propose a WSR-MGAT model based on meta-path-guided graph attention networks, utilizing graph embeddings to mine structural information in Web services. Additionally, Zhang et al. [27] introduce a collaborative tag-aware graph neural network for long-tail service recommendation. This approach fully utilizes information contained in heterogeneous and homogeneous networks. Moreover, Yu et al. [28] utilize GraphGAN to obtain embeddings for Mashups and services in the Mashup-service-tag graph, thereby improving generalization accuracy to a certain extent. Even though user and service features could be embedded by network representation learning from service networks, many works only focus on structural semantics while neglecting the textual semantics.

3) Hybrid Approaches. To fully utilize the textual and structural semantics, hybrid-based approaches extract both textual and structural semantics from functional description documents and service networks to achieve service recommendation [29], [30], [31]. These approaches show great improvement in service recommendation. Thus, they are the state-of-the-art approaches for API recommendation. More specifically, Wu et al. [30] propose a recommendation method that integrates multiple model fusion and multi-task learning. This approach employs convolutional neural networks as semantic components and utilizes matrix factorization techniques to capture graph structure information. Additionally, Yu et al. [31] introduce a hybrid service recommendation method SRCLML, which considers both content-based and graph-based information. This method incorporates multi-task learning to enhance model performance. Meanwhile, Wang et al. [32] present a recommendation model FSFM. This model extracts structural information from heterogeneous graphs and combines it with textual information for service recommendation.

Even though hybrid-based approaches make big progress in service recommendation compared with content-based approaches and graph-based approaches, there is still a problem that the rich semantics cannot be extracted sufficiently from service description documents and service networks simultaneously, resulting in limited recommendation performance, since the representation processes of functional description documents and service networks are conducted independently in existing works. Thus, how to jointly represent the textual and structural semantics simultaneously to derive the proper embeddings for effective service recommendation remains an open issue, which is the motivation of our work.

III. Framework

Give the set of Web APIs $\mathbb{A} = \{A_1, A_2, \ldots, A_M\}$, and the set of Mashups $\mathbb{M} = \{M_1, M_2, \ldots, M_N\}$, where each

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

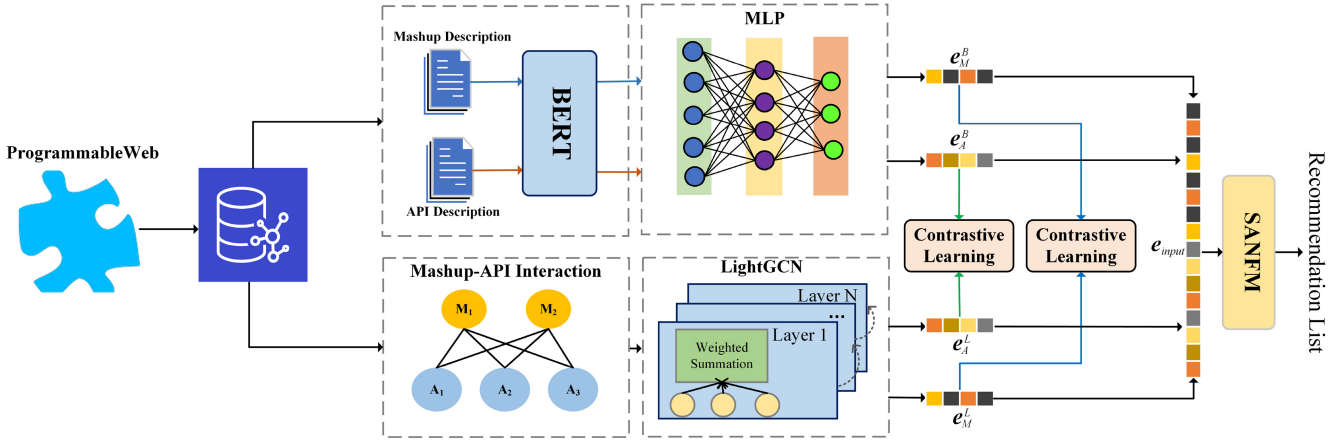


Fig. 1. The framework of Web API recommendation via exploring textual and structural semantics with contrastive learning and joint training.

Mashup is a set of Web APIs. And, usually each Web API or Mashup has its functional description document. Web API recommendation is to provide a list of Web APIs according to the Mashup requirement given by the active user. To promote the Mashup development for users, this paper proposes an effective Web API recommendation approach via exploring textual and structural semantics with contrastive learning and joint training. The overall framework of the proposed approach is illustrated in Fig. 1, while the more specific details of the main components will be described in Section IV.

In the framework, the Web API and Mashup data are crawled from Web API markets, like ProgrammableWeb. The involved data include functional description documents and interaction relationships between Mashups and Web APIs. Based on the functional description documents of Web APIs or Mashups, their initial textual embeddings are generated with the BERT model. Meanwhile, with the interaction relationships between Mashups and Web APIs, the Mashup-API interaction graph is constructed, by which the structural embeddings of Mashups and Web APIs are derived with the LightGCN model. To correlate the representation process of textual and structural embedding, two contrastive learning components are employed for Web API and Mashup embedding, respectively. Prior to this, the initial textual embeddings undergo dimensionality reduction and semantic alignment with the structural embeddings through a simple MLP (Multi-Layer Perceptron). By contrastive leaning through cross-view, optimized textual and structural embeddings could be derived. Afterward, the derived textual and structural embeddings are concatenated as aggregated vectors, which are taken as the input of the downstream Web API recommendation model SANFM that has strong expressiveness by modelling low and high order feature interactions with the diverse importance of feature interactions. In SANFM, the input features could be deeply fused to further derive the accurate Web API recommendation list. In our framework, the representation process of the textual and structural embedding and service recommendation process are trained jointly. Thus, the update of the parameters in the whole framework could be more effective, since they interact with each other to achieve the common objective, i.e., the final joint loss function.

IV. Web API Recommendation via Exploring Textual and Structural Semantics

In this section, we introduce the components of the proposed Web API recommendation framework. Firstly, the textual representation by the BERT model is introduced. Meanwhile, the structural representation by the LightGCN model is presented. Then, contrastive learning is designed to optimize textual and structural representation. Finally, the deep learning-based service recommendation model, i.e., SANFM, is introduced, and the overall optimization function is described.

A. Textual Representation Based on BERT

BERT (Bidirectional Encoder Representations from Transformers) is an NLP (Natural Language Processing) model proposed by Google in 2018 [12]. The emergence of BERT has directly promoted the development of NLP. Thus, it has been widely used in various NLP tasks, such as question-answering systems and text classification. BERT has two outstanding characteristics as follows. 1) *Transformer Architecture*. BERT is a pre-training model based on transformer architecture. It adopts a bidirectional model and can consider context information instead of one-way context during pre-training, so it can better understand the meaning and semantic relationship of words. 2) *Large-scale Pre-training*. BERT learns a common language representation from a large amount of text data through unsupervised learning, so that it can obtain more semantic information, better understand and process natural language, and adapt to a variety of applications.

Taking a sentence as input, the BERT model intricately captures the essence of each word by incorporating three embedded characteristics: *Token Embedding*, *Segment Embedding*, and *Position Embedding* [33], as shown in Fig. 2. **Token Embedding** creates a distinctive token sequence for each word. This distinctive sequence forms the bedrock for BERT's adept understanding of individual word semantics and contributions to the overall sentence meaning. **Segment**

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

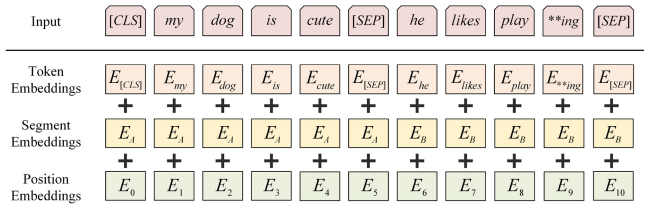


Fig. 2. An example input representation by BERT.

Embedding sheds light on the sentence context to which a word belongs. This contextual awareness enhances BERT's proficiency in capturing the relationships between words in a sentence, contributing to a holistic understanding of the input. **Position Embedding** encodes the precise position of individual words into a set of vectors. This positional information refines BERT's comprehension, allowing it to discern the sequential order and hierarchical structure within the sentence. The three embeddings mentioned above will be summed as the input to the BERT model, resulting in a vector group representing each word.

In our work, we input the functional description documents of APIs and Mashups into the BERT model to derive the initial textual embeddings for services. The derived embeddings are composed of token embeddings, segment embeddings, and position embeddings. Afterward, we adopt the simple MLP model to achieve further transformation of the representations obtained by the BERT model. Specifically, after the BERT component, a nonlinear connection layer is added. MLP consists of three layers, i.e., input layer, hidden layer, and output layer. Assuming that the input vector is represented by $\boldsymbol{x}$, the output $f(\boldsymbol{x})$ of the hidden layer is defined as Formula (1).

$$\boldsymbol{e}^B = \sigma_2(\boldsymbol{W}_2(\sigma_1(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2)) \tag{1}$$

where $\mathbf{W}$, $\mathbf{b}$, and σ represent the weights, bias, and activation function, $\boldsymbol{e}^B$ denotes the final textual embedding. Thus, the final textual embeddings of Mashup M_i and Web API A_i are denoted as $\boldsymbol{e}^B_{M_i}$ and $\boldsymbol{e}^B_{A_i}$, respectively. The design of the nonlinear connection layer facilitates the flexible transformation for the further optimization of textual representation driven by the following contrastive leaning.

B. Structural Representation Based on LightGCN

GCN (Graph Convolutional Network) is one of the widely used graph convolutional networks, which extracts the features of nodes in a graph based on structure information [3]. Considering that feature transformations and nonlinear activations in GCNs may have little contribution to the performance improvement in the collaborative filtering-based recommendation [23], LightGCN is employed to derive structural representations from the Mashup-API interaction graph. LightGCN mainly consists of three layers: *Embedding Layer*, *Propagation Layer*, and *Weighted Layer Combination*. The framework of LightGCN is shown in Fig. 3.

Embedding Layer. At the embedding layer, Mashups and APIs are converted into vectors through initialization. Mashup M_i is represented by the embedding vector $\boldsymbol{e}^L_{M_i}$, and Web API A_i is represented by the embedding vector $\boldsymbol{e}^L_{A_i}$. Thus, the parameter matrix $\boldsymbol{E}^L$ in Formula (2) could be a look-up table for service embeddings. The embedding will be further refined at the propagation layer to obtain the final effective embedding vectors through joint training with the downstream task.

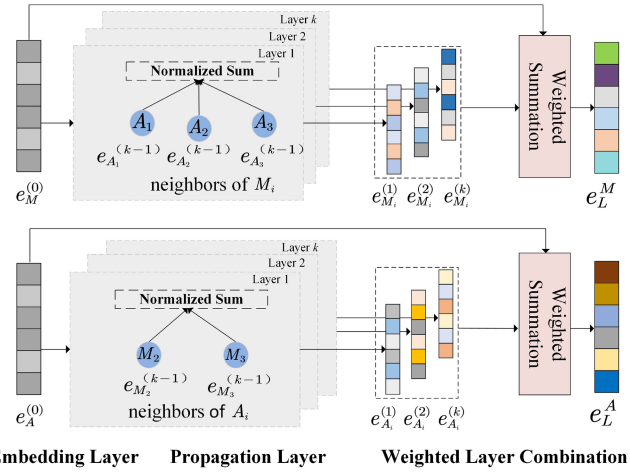


Fig. 3. The framework of LightGCN.

$$\boldsymbol{E}^L = \left[\boldsymbol{e}^L_{M_1}, \boldsymbol{e}^L_{M_2}, \ldots, \boldsymbol{e}^L_{M_N}; \boldsymbol{e}^L_{A_1}, \boldsymbol{e}^L_{A_2}, \ldots, \boldsymbol{e}^L_{A_M}\right] \tag{2}$$

Propagation Layer. At the propagation layer, the graph convolution operation (i.e., propagation rule) in LightGCN is defined as Formula (3) and (4), where $\frac{1}{\sqrt{|\mathcal{N}_A|}\sqrt{|\mathcal{N}_M|}}$ is a symmetric normalized term to avoid the scale of embeddings increasing with graph convolution operations, $\mathcal{N}_M$ and $\mathcal{N}_A$ represent the neighbors of Mashup M and Web API A, $\mathbf{e}^{(k)}_A$ is the kth layer embedding of Web API A, $\mathbf{e}^{(k)}_M$ is the kth layer embedding of Mashup M.

$$\mathbf{e}^{(k+1)}_M = \sum_{A \in \mathcal{N}_M} \frac{1}{\sqrt{|\mathcal{N}_M|}\sqrt{|\mathcal{N}_A|}} \mathbf{e}^{(k)}_A \tag{3}$$

$$\mathbf{e}^{(k+1)}_A = \sum_{M \in \mathcal{N}_A} \frac{1}{\sqrt{|\mathcal{N}_A|}\sqrt{|\mathcal{N}_M|}} \mathbf{e}^{(k)}_M \tag{4}$$

Weighted Layer Combination. The weighted summation and aggregation operators are employed to integrate the embeddings from different layers for both Mashups and Web APIs, so that the final embedding is derived, defined as Formula (5) and (6). Specifically, the final structural embeddings of Mashup M_i and Web API A_i are denoted as $\boldsymbol{e}^L_{M_i}$ and $\boldsymbol{e}^L_{A_i}$.

$$\mathbf{e}^L_M = \sum_{k=0}^{K} \frac{1}{K+1} \mathbf{e}^{(k)}_M \tag{5}$$

$$\mathbf{e}^L_A = \sum_{k=0}^{K} \frac{1}{K+1} \mathbf{e}^{(k)}_A \tag{6}$$

To derive the trained LightGCN model, the BPR (Bayesian Personalized Ranking) loss [34] is employed, which is a pairwise loss that encourages the prediction of an observed

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

entry to be higher than its unobserved counterpart, defined as Formula (7).

$$L_{BPR} = \sum_{(M_k, A_i, A_j) \in \mathbb{S}} -ln\sigma\left(\hat{y}_{(M_k, A_i)} - \hat{y}_{(M_k, A_j)}\right) \quad (7)$$

where $\mathbb{S} = \{(M_k, A_i, A_j) | (M_k, A_i) \in \mathbb{S}^+, (M_k, A_j) \in \mathbb{S}^-\}$ denotes the pairwise training data, $\mathbb{S}^+$ is the observed interactions (i.e., positive samples), and $\mathbb{S}^-$ is the unobserved interactions (i.e., negative samples).

C. Contrastive Learning Between Textual and Structural Representations

Contrastive learning encourages features from different views to come closer in the embedding space, thereby achieving self-distinguishing feature representation in an unsupervised manner and supplementing supervised tasks with unlabeled data [35]. In our work, we apply cross-view contrastive learning to align the textual representations from BERT to the structural representations from LightGCN since they both extract the functional semantics of Mashups and Web APIs from different views. InfoNCE loss [36] is usually used for the contrastive learning loss defined as Formula (8) and (9), which minimize the distance between positive samples while maximizing the distance between negative samples.

$$L_M^{CL} = -\frac{1}{|\mathbb{M}|} \sum_{M_i \in \mathbb{M}} log \frac{exp\left(s\left(\mathbf{e}_{M_i}^B, \mathbf{e}_{M_i}^L\right)/\tau\right)}{\sum_{M_k \in \mathbb{M}} exp\left(s\left(\mathbf{e}_{M_i}^B, \mathbf{e}_{M_k}^L\right)/\tau\right)} \quad (8)$$

$$L_A^{CL} = -\frac{1}{|\mathbb{A}|} \sum_{A_i \in \mathbb{A}} log \frac{exp\left(s\left(\mathbf{e}_{A_i}^B, \mathbf{e}_{A_i}^L\right)/\tau\right)}{\sum_{A_k \in \mathbb{A}} exp\left(s\left(\mathbf{e}_{A_i}^B, \mathbf{e}_{A_k}^L\right)/\tau\right)} \quad (9)$$

where $s(\cdot, \cdot)$ is discriminator that computes the similarity between two vectors with consine similarity, and τ is a temperature parameter. The size of the temperature parameter directly affects the effectiveness of contrastive learning, particularly in ensuring the uniformity of sample feature distributions. Thus, it should be set appropriately according to experiments. By performing Formula (8) and (9), the cross-view contrastive learning integrates both structural and textual semantics to achieve more effective feature extraction. Subsequently, the textual and structural embeddings of the Mashup and the Web API are concatenated to derive the overall embedding $\boldsymbol{e}_{input}$, defined as Formula (10), which serves as the input to the downstream Web API recommendation model.

$$\boldsymbol{e}_{input} = \boldsymbol{e}_M^B \Big\| \boldsymbol{e}_M^L \Big\| \boldsymbol{e}_A^B \Big\| \boldsymbol{e}_A^L \quad (10)$$

D. Web API Recommendation Model Based on Self-Attentional Neural Factorization Machines

BasicFM [37] is a machine learning model based on factorization machines, combining the advantages of linear models and matrix factorization for modeling interactions among embedding vectors. However, due to its limitation in modeling interactions of higher-order interaction features, BasicFM may introduce noise, especially when combining unrelated features. Therefore, we propose to use SANFM [11], which is the state-of-the-art model for Web API recommendation, to accurately model and analyze the textual and structural semantics of Mashups and Web APIs, defined as Formula (11).

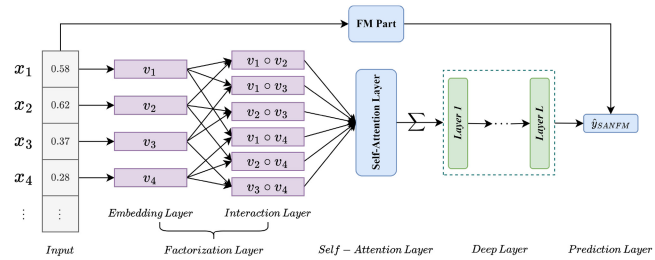


Fig. 4. The neural network architecture of SANFM.

$$\hat{y}_{SANFM}(\boldsymbol{x}) = \boldsymbol{w}_0 + \sum_{i=1}^{n} \boldsymbol{w}_i \boldsymbol{x}_i + h(\boldsymbol{x}) \quad (11)$$

where the first two terms represent global bias and linearity which is denoted as $\hat{y}_{Linear}(\boldsymbol{x})$, and the last term $h(\boldsymbol{x})$ is the core of SANFM. The neural network architecture of SANFM is illustrated in Fig. 4. As can be seen from the figure, SANFM is composed of four layers: *Factorization Layer*, *Self-Attention Layer*, *Deep Layer*, and *Prediction Layer*.

Factorization Layer. This layer aims to convert the sparse features of the input into a low-dimensional dense vector. Typically, it utilizes matrix factorization techniques to capture the latent characteristics of Mashups and Web APIs. Consequently, we represent the vector as $\boldsymbol{v}_x = \{x_1\boldsymbol{v}_1, \ldots, x_n\boldsymbol{v}_n\}$ with $\boldsymbol{x}$. Subsequently, these vectors are forwarded to the interaction layer for modeling, as defined in Formula (12), where $\odot$ denotes the element-wise product of two vectors, that is, $(\boldsymbol{v}_i \odot \boldsymbol{v}_j)_k = v_{ik} v_{jk}$.

$$\boldsymbol{Z}_x = \sum_{i=1}^{n} \sum_{j=i+1}^{n} x_i \boldsymbol{v}_i \odot x_j \boldsymbol{v}_j \quad (12)$$

Self-Attention Layer. The self-attention layer is adept at capturing intricate relationships between Mashups and APIs, as defined in Formula (13). By computing self-attention weights among each element and others, this mechanism discerns interaction patterns and dependencies between Mashups and APIs, enhancing the comprehension of user preferences.

$$\boldsymbol{Z}_x = Attention(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = soft\max\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\boldsymbol{T}}}{\sqrt{d_k}}\right)\boldsymbol{V} \quad (13)$$

where $\boldsymbol{Z}_x$ denotes the pairwise interactions with self-attention. $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ represent the Query, Key, and Value Matrices, respectively. These matrices are utilized to calculate attention weights between features, facilitating the learning of feature interactions. The degree of correlation between $\boldsymbol{Q}$ and $\boldsymbol{K}$ determines the significance of $\boldsymbol{V}$ in attention weighting, thereby influencing the ultimate representation of feature interactions.

Deep Layer. To better capture the high-order relationships between features, we introduce a deep neural network layer. This layer utilizes multiple layers of neurons and nonlinear

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

transformations to further extract high-order feature interactions, enabling more fine-grained feature learning, as defined in Formula (14).

$$\begin{aligned} \boldsymbol{o}_1 &= \sigma_1(\boldsymbol{W}_1\boldsymbol{Z}_x + \boldsymbol{b}_1) \\ \boldsymbol{o}_2 &= \sigma_2(\boldsymbol{W}_2\boldsymbol{o}_1 + \boldsymbol{b}_2) \\ &\vdots \\ \boldsymbol{o}_L &= \sigma_L(\boldsymbol{W}_L\boldsymbol{o}_{L-1} + \boldsymbol{b}_L) \end{aligned} \tag{14}$$

where $\boldsymbol{o}_i$ denote the output of the ith layer, σ_i is the activation function used to introduce nonlinear properties. Common activation functions include *Sigmoid*, *ReLU*, *Tanh*, etc. $\boldsymbol{W}_i$ is the weight matrix of the ith layer, which is used to transform the output of the previous layer into the input of the current layer. And $\boldsymbol{b}_i$ is the bias term of the ith layer, which is used to adjust the input data of each layer. Finally, to obtain the predicted probability, we also include the output of the last hidden layer as part of the calculation, as defined in Formula (15) and (16), where L is the number of layers in the Deep Layer and $\boldsymbol{h}$ is the neural weight in the prediction layer.

$$h(\boldsymbol{x}) = \boldsymbol{h}^T\boldsymbol{o}_L \tag{15}$$

$$h(\boldsymbol{x}) = \boldsymbol{h}^T\sigma_L(\boldsymbol{W}_L(\cdots(\boldsymbol{W}_1\boldsymbol{Z}_x + \boldsymbol{b}_1)\cdots) + \boldsymbol{b}_L) \tag{16}$$

Prediction Layer. We consider the task of the SANFM module as a binary classification problem, where for a given Mashup and Web API, only two states exist: recommending or not recommending, as defined in Formula (17).

$$\hat{y}(\boldsymbol{x}) = Sigmoid(\hat{y}_{SANFM}(\boldsymbol{x})) \tag{17}$$

Finally, we utilize cross-entropy as the loss function for SANFM in Formula (18), where N is the number of training samples, $y^{(i)}$ is the actual value and $\widehat{y}^{(i)}$ is the predicted value. We employ stochastic gradient descent to optimize the loss function. During the prediction phase, the final output consists of a list of the Top-K Web APIs, sorted in descending order based on the predicted probabilities.

$$L_{rec} = -\frac{1}{N}\sum_{i=1}^{N}\left[y^{(i)}\log\left(\widehat{y}^{(i)}\right) + \left(1 - y^{(i)}\right)\left(\log\left(1 - \widehat{y}^{(i)}\right)\right)\right] \tag{18}$$

E. Optimization

By aggregating the task loss functions of the sub-modules above and imposing weight constraints, we derive a comprehensive joint training objective function as defined in Formula (19) that includes BPR loss from LightGCN, InfoNCE loss from contrastive learning, and cross-entropy loss from SANFM. The design of this objective function aims to achieve effective balance among the tasks of each module and to maintain reasonable parameter adjustments during model training, thereby enhancing the robustness and generalization capability of the model.

$$L = L_{BPR} + \gamma\left(L_M^{CL} + L_A^{CL}\right) + L_{rec} + \lambda\|\Theta\|^2 \tag{19}$$

where γ is the parameter to control the contrastive loss, Θ represents the trainable parameter set of the whole model, λ is the weight of L_2 regularization aimed at mitigating overfitting, L_{BPR} and L_{rec} denote the loss functions of LightGCN and SANFM modules, respectively. Through the combined loss function, the proposed approach can effectively enhance interactions to achieve the common objective of recommendation.

TABLE I
CHARACTERISTICS OF THE DATASET

Item	Value
Number of Mashups	5, 751
Number of Web APIs	1, 188
Number of Web APIs per Mashup	1.86

V. EXPERIMENTS

In this section, we describe the experimental setup in Section V-A, including *Dataset Description*, *Evaluation Metrics*, and *Baseline Methods*. Then, we present the experimental results to evaluate the proposed approach in Section V-B, including *Compare with Baseline Methods*, *Ablation Analysis*, *Impact of Hyper-parameters*.

A. Experimental Setup

Dataset Description. The dataset are collected from ProgrammableWeb in February 2022, including 22,576 Web APIs and 6,454 Mashups, which could be available online.[3] To obtain the textual representations of Mashups and Web APIs, the ones that lack description data are removed. Meanwhile, to obtain the structural representations of Mashups and Web APIs, only the Web APIs that are used before are considered in the preprocessed dataset. Consequently, there are 1,188 Web APIs and 5,751 Mashups in the final preprocessed dataset, as illustrated in Table I. And, the average number of Web APIs per Mashup is 1.86. The dataset is randomly partitioned into training set and testing set at the ratio of 8:2.

Evaluation Metrics. We evaluate the proposed model using five widely adopted evaluation metrics in service recommendation: *Precision*, *Recall*, *F1 Score*, *MAP*, and *NDCG*.

Precision. Precision is the ratio of the number of true hits in the recommended Web API list to the total number of recommended Web APIs, as defined in Formula (20), where *TopKAPIs* and *RealAPIs* represent the recommended API list and the actual API list with invocation relationships.

$$Precision@K = \frac{|TopKAPIs \bigcap RealAPIs|}{K} \tag{20}$$

Recall. Recall assesses the model's capability in identifying genuine Web APIs within the recommended API list by quantifying the proportion of true hits to the total number of actual APIs, as defined in Formula (21).

$$Recall@K = \frac{|TopKAPIs \bigcap RealAPIs|}{|RealAPIs|} \tag{21}$$

F1 Score. F1 Score is the harmonic mean of Precision and Recall, serving as a comprehensive metric to better reflect the performance of the service recommendation model, as defined

[3]https://github.com/IntelligentServiceLab/Web-Service-Crawler

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

in Formula (22). Generally, a higher F1 Score indicates the better performance.

$$F1@K = \frac{2 \times Precision@K \times Recall@K}{Precision@K + Recall@K} \tag{22}$$

MAP. Mean average precision provides a comprehensive measure of precision at different positions in the recommendation list, as defined in Formula (23) and (24), where $I(i)$ is 1 or 0 indicating whether the Web API at the ith position in the recommendation list is an actual Web API used by the target Mashup M_i. AP_{M_i} represents the average precision of Mashup M_i for different Top-K recommendation results, and MAP is the average of AP values for all the Mashups in the testing set $\mathbb{M}_{test}$.

$$AP_{M_i}@K = \frac{1}{|M_i|}\sum_{i=1}^{K}(Precision@i \times I(i)) \tag{23}$$

$$MAP@K = \frac{1}{|\mathbb{M}_{test}|}\sum_{M_i \in \mathbb{M}_{test}} AP_{M_i}@K \tag{24}$$

NDCG. Normalized discounted cumulative gain is a ranking quality metric. It compares the recommended ranking to an ideal order where all relevant items are at the top of the list, defined by Formula (27). *NDCG@K* is determined by dividing the Discounted Cumulative Gain (DCG) by the ideal DCG (IDCG) representing a perfect ranking, where *DCG* and *IDCG* are defined as Formula (25) and (26), respectively.

$$DCG@K = \sum_{i=2}^{K} \frac{2^{I(i)} - 1}{\log_2(i+1)} \tag{25}$$

$$IDCG@K = \sum_{i=1}^{|M_i|} \frac{1}{\log_2(i+1)} \tag{26}$$

$$NDCG@K = \frac{DCG@K}{IDCG@K} \tag{27}$$

Baseline Methods. We compare the proposed approach CLJT with the following categories of approaches: (1) Content-based Approaches (DeepFM [5] and NAFM [6]), (2) Graph-based Approaches (NGCF [22], BPRMF [38], and LightGCN [23]), (3) Hybrid Approaches (SRCLML [31]). The baseline methods are described as follows.

- *DeepFM [5]:* The deep neural network factorization machine captures both low-order and high-order feature relationships, while also learning complex nonlinear feature interactions, enabling efficient service recommendation.
- *NAFM [6]:* The neural and attentional factorization machine integrates deep neural networks to model nonlinear feature interactions and attention mechanisms to discern the varying importance among interacting features.
- *NGCF [22]:* Neural graph collaborative filtering combines graph neural networks with collaborative filtering techniques. It leverages graph neural networks to represent nodes of Mashup and Web APIs and utilizes collaborative filtering to capture their similarities, thereby enabling effective service recommendations.

TABLE II
DEFAULT VALUES OF HYPER-PARAMETERS

Item	Value
Embedding size d_l in LightGCN	64
Embedding size d_s in SANFM	128
Number of propagation layers in LightGCN	3
Learning rate	0.005
Temperature τ	0.0015
Contrast Learning loss rate γ	0.0005
Dropout rate	0.5
L_2 regularization strength λ	0.0001
Top-K	5

- *BPRMF [38]:* Bayesian personalized ranking matrix factorization achieves personalized service recommendation by learning the feature vectors of users and items, and employing Bayesian inference for personalized ranking.
- *LightGCN [23]:* Simplifying and powering graph convolution network streamlines the classical GCN framework by eliminating feature transformation and nonlinear activation operations. It employs convolutional learning on the representations of user and item nodes to enable service recommendations.
- *SRCLML [31]:* This approach utilizes Transformer to extract features from the textual descriptions of Mashups and Web APIs, and employs LightGCN to extract structural information from their interaction history. Finally, it integrates the information from both perspectives to improve service recommendation.

B. Performance Evaluation

Before reporting the experimental results, we present the default values of hyper-parameters in the proposed model as illustrated in Table II. The default hyper-parameter values are derived by model optimization through grid search under the preprocessed dataset. In particular, it is worth stating that since the average number of associated Web APIs in a Mashup is only 1.86 as indicated in Table I, we set the value of Top-K as 3, 5, and 7 in our experiments, and the default value as 5. Thus, a larger value of top-k is less meaningful to users.

Compare with Baseline Methods. We compare the performance of the proposed method CLJT with the baseline methods mentioned in Section V-A, as demonstrated in Table III. Due to the sparsity of Mashup-API invocation matrix, the overall performance of service recommendation is generally low. It can be observed from the table that the methods, such as DeepFM, NAFM, and NGCF, exhibit relatively poor performance across all metrics, since DeepFM and NAFM primarily utilize the textual embeddings from Mashup and API descriptions to capture latent information without considering the service networks and NGCF mainly focuses on Mashup-API invocations while neglecting the textual descriptions. LightGCN outperforms BPRMF and NGCF. This is because BPRMF, based on matrix factorization, experiences rapid performance degradation when handling large-scale data and complex relationships, making it challenging to capture higher-order and implicit relationships among services. And NGCF is a collaborative filtering recommendation model based on the graph convolutional neural network GCN. In contrast, LightGCN makes improvement upon GCN and leverages

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE COMPARISON OF DIFFERENT WEB API RECOMMENDATION METHODS

K	Method	Precision	Recall	F1 Score	MAP	NDCG
3	DeepFM [5]	0.02083	0.03125	0.02500	0.03125	0.03832
	NAFM [6]	0.04167	0.01250	0.01923	0.00625	0.03701
	NGCF [22]	0.03857	0.11571	0.05786	0.07891	0.08831
	BPRMF [38]	0.09837	0.14756	0.11805	0.20391	0.22034
	LightGCN [23]	0.09581	0.23734	0.13652	0.16964	0.19396
	SRCLML [31]	0.14453	0.19842	0.16724	0.20356	0.24122
	CLJT	**0.25521**	**0.24065**	**0.24771**	**0.23872**	**0.30011**
	Improv.	76.58%	1.39%	48.12%	17.07%	24.41%
5	DeepFM [5]	0.02500	0.12500	0.04167	0.12500	0.12500
	NAFM [6]	0.02500	0.03125	0.02778	0.00781	0.02102
	NGCF [22]	0.02909	0.14544	0.04848	0.08570	0.10056
	BPRMF [38]	0.07176	0.17941	0.10252	0.19851	0.24521
	LightGCN [23]	0.07454	**0.30867**	0.12008	0.19036	0.22601
	SRCLML [31]	0.10547	0.23335	0.14527	0.19986	0.24714
	CLJT	**0.19687**	0.30244	**0.23849**	**0.21969**	**0.28791**
	Improv.	86.60%	-2.01%	64.17%	10.67%	16.50%
7	DeepFM [5]	0.02083	0.01562	0.01786	0.01562	0.02933
	NAFM [6]	0.01250	0.06250	0.02083	0.06250	0.06250
	NGCF [22]	0.02442	0.17091	0.04273	0.08975	0.10944
	BPRMF [38]	0.05808	0.20329	0.09035	0.19099	0.26062
	LightGCN [23]	0.05810	0.33457	0.09900	0.19528	0.23612
	SRCLML [31]	0.07991	0.22609	0.11808	0.17951	0.22977
	CLJT	**0.14192**	**0.35885**	**0.20339**	**0.20290**	**0.30183**
	Improv.	77.60%	7.00%	72.25%	4.00%	15.81%

graph neural network techniques to better learn user and item representations. Consequently, LightGCN is more effective in capturing the complex relationships among services, thus exhibiting superior performance. The SRCLML method surpasses the aforementioned approaches as it comprehensively considers both textual descriptions and structural information of Mashups and Web APIs, while incorporating multi-task learning for better model optimization. Even so, the proposed approach CLJT demonstrates significant superiority due to introducing cross-view contrastive learning and joint training so that CLJT can effectively integrate both textual and structural information to enhance model performance. Typically, under the condition of Top-K=3, compared to the best results among the baseline methods, CLJT method improves Precision, Recall, F1 Score, MAP, and NDCG by 76.58%, 1.39%, 48.12%, 17.07% and 24.41%, respectively. Overall, graph-based approaches show slightly better performance than content-based approaches. And hybrid approaches show much better performance than both content-based approaches and graph-based approaches. Therefore, the above observations and analysis verify the effectiveness of the proposed approach.

Ablation Analysis. We conduct ablation experiments to study the effectiveness of the components in the designed approach. Four main components are considered: *BERT*, *LightGCN*, *contrastive learning* (CL), *joint training* (JT). By removing each of them, four variants are derived, as follows.

- *CLJT-BERT:* This method is derived by removing the textual representation based on BERT, in which only the structural representation is used to explore the relationship between Mashups and Web APIs. Thus, contrastive learning is also removed.
- *CLJT-LightGCN:* This method is derived by removing the structural representation based on LightGCN, in which only the textual representation is used to explore the relationship between Mashups and Web APIs. Thus, contrastive learning is also removed.

TABLE IV
ABLATION ANALYSIS

Method	Precision	Recall	F1 Score	MAP	NDCG
CLJT-BERT	0.17344	0.22393	0.19547	0.17599	0.24149
CLJT-LightGCN	0.18906	0.25500	0.21713	0.18561	0.25718
CLJT-CL	0.18437	0.26525	0.21753	0.19702	0.26916
CLJT-JT	0.18852	0.28079	0.22558	0.20313	0.28146
CLJT	**0.19687**	**0.30244**	**0.23849**	**0.21969**	**0.28791**

- *CLJT-CL:* This method is derived by removing the contrastive learning components between the textual representation and the structural representation for Mashups and Web APIs. Thus, the textual and the structural representations are directly used as inputs to SANFM for Web API recommendation.
- *CLJT-JT:* This method is derived by removing the joint training mechanism, in which the textual and structural representation learning and the recommendation model are trained independently.

The results of ablation experiments are shown in Table IV. As can be seen from the table, the contrastive learning module integrates both textual and graph structural information, resulting in superior performance. Meanwhile, joint training facilitates a more effective balance among module tasks. Besides, the of recommendation performance based solely on textual or graph structural information is inferior to that of methods considering both types of information comprehensively. Therefore, generally, CLJT-CL and CLJT-JT outperform CLJT-BERT and CLJT-LightGCN. Comparatively, CLJT-LightGCN is better than CLJT-BERT, indicating that textual information contributes more than structural information. And CLJT-JT has slight better performance than CLJT-CL, indicating that contrastive learning makes more contribution to the improvement of service recommendation compared with the joint training mechanism in the proposed method. Overall, CLJT always remains the best performance.

Impact of Hyper-parameters. In the proposed Web API recommendation approach, there are mainly four hyper-parameters involved, i.e., the embedding size d_l in LightGCN, the embedding size d_s in SANFM, the temperature parameter τ in contrastive learning, and the controlling weight γ of contrastive learning loss in the joint training objective. Therefore, A series of experiments are conducted on the dataset to investigate the effect of different hyper-parameters on the model proposed in this paper. All experiments below are conducted under the condition of Top-K=5.

1) The effect of d_l: To investigate the impact of LightGCN embedding size d_l on the model performance, we vary d_l within the range of {16,32, . . . , 256}. As shown in Fig. 5, all the evaluation metrics exhibit similar trends as the embedding size rises from 16 to 256. It can be observed that the model performance gradually increases as d_l increases first, and get the peak point at $d_l = 64$. However, as d_l continues to increase, the model performance begins to decline. When the embedding size is too small, it restricts the potential structural features that can be captured, leading to relatively poor performance. In contrast, when the embedding size is too large, the complexity of the model increases, making it prone

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

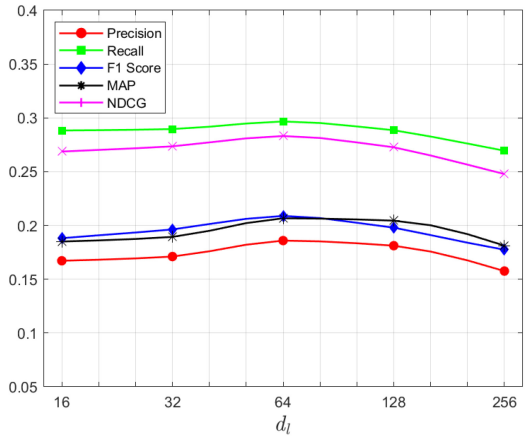


Fig. 5. The effect of d_l (i.e., embedding size in LightGCN).

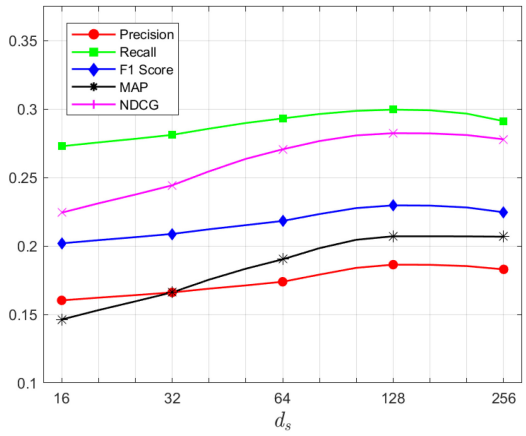


Fig. 6. The effect of d_s (i.e., embedding size in SANFM).

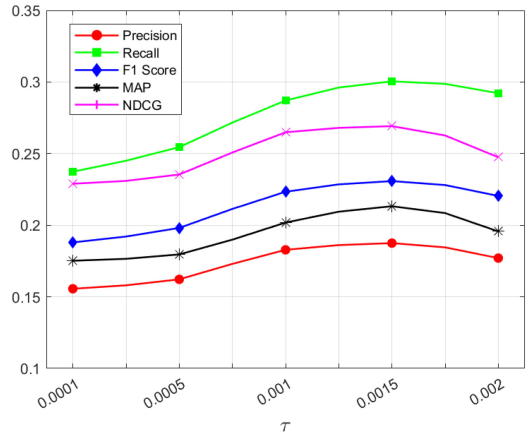


Fig. 7. The effect of τ (i.e., temperature parameter in contrastive learning).

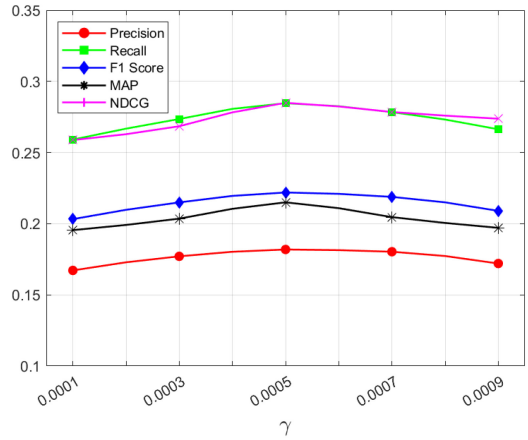


Fig. 8. The effect of γ (i.e., controlling weight of contrastive learning loss).

to overfitting and introducing noise. Thus, 64 is chosen as the default value for the embedding size in LightGCN.

2) The effect of d_s: The embedding size d_s in SANFM is studied by varying it within the range of {16, 32, ..., 256} to estimate the impact on the whole model performance. Fig. 6 shows the performance change of the model as the embedding size increases. The SANFM model integrates self-attention layer and deep neural network layer. Thus, a smaller embedding size may restrict the model's expressive capacity, making it challenging to handle complex data patterns. That is to say, an appropriate embedding size is significant. As can be seen from the figure, the model performance achieves its peak when $d_s = 128$. Thus, we establish the default value of the embedding size in SANFM as 128.

3) The effect of τ: The temperature parameter τ is pivotal in contrastive learning, directly influencing its efficacy, especially in maintaining uniformity across sample feature distributions. A smaller τ enhances contrastive learning by reducing distances between similar samples and increasing distances between dissimilar ones, thereby enhancing the discernment of sample similarities. However, extreme values of τ may disrupt the uniformity of sample feature distributions, impacting the effectiveness of contrastive learning. In the experiments, we explore a series of different values of τ in a large range at the beginning. And then, based on the observation of the coarse-grained experimental results, we focus on the small interval of τ that could demonstrate better performance on the model performance. Specifically, we investigate within the range of {0.0001, 0.0005, ..., 0.002}. As shown in Fig. 7, the model achieves the optimal performance when $\tau = 0.0015$. Nevertheless, as τ increases from 0.0015, the model's ability to detect negative samples diminishes, leading to a decline in the effectiveness of contrastive learning.

4) The effect of γ: We adjust the controlling weight γ of the contrastive loss within the range of {0.0001, 0.0003, ..., 0.0009} to explore the impact on service recommendation performance, as shown in Fig. 8. The results indicate a slight improvement in all metrics as γ increases, highlighting the effectiveness of the contrastive learning. The model performance gets the peak at $\gamma = 0.0005$ before declining. It is suggested that excessively large controlling weight during joint training may overly emphasize the contrastive loss, potentially misleading representation learning for applications and services, thereby causing a decrease in the final model performance. Therefore, 0.0005 is set as the optimal value for the controlling weight γ.

VI. Conclusion

To improve the performance of Web API recommendation, this paper proposes an effective Web API recommendation

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

approach via exploring textual and structural semantics with contrastive learning and joint training, named CLJT. CLJT utilizes both textual and structural semantics simultaneously for Web API recommendation. BERT and LightGCN are employed to extract textual and structural semantics for Mashups and Web APIs, respectively. Moreover, through cross-view contrastive learning, the textual and structural features are correlated with each other to improve their representations. Finally, a deep learning-based model SANFM is used for training the Web API recommendation model. In the whole learning process, joint training is proposed to balance the tasks of each module so that the recommendation model could be improved further. Extensive experiments are conducted over a real-world dataset and the experimental results demonstrate the superior performance of the proposed method compared with the state-of-the-art baselines. And ablation study verifies the significance of the designed contrastive learning and joint training mechanisms. However, the data sparsity problem in Web API recommendation is still not solved. In future work, we will explore the multi-view and multi-level contrastive learning to alleviate the data sparsity problem more effectively.

References

[1] R. Zhang et al., "Privacy-aware Web APIs recommendation for consumer mashup creation based on iterative quantification," *IEEE Trans. Consum. Electron.*, early access, Mar. 5, 2024, doi: 10.1109/TCE.2024.3372572.

[2] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.

[3] B. Jiang, Z. Zhang, D. Lin, J. Tang, and B. Luo, "Semi-supervised learning with graph learning-convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11305–11312.

[4] X. Zhang, J. Liu, B. Cao, Q. Xiao, and Y. Wen, "Web service recommendation via combining Doc2Vec-based functionality clustering and deepFM-based score prediction," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl.*, 2018, pp. 509–516.

[5] M. Ma, G. Wang, and T. Fan, "Improved deepFM recommendation algorithm incorporating deep feature extraction," *Appl. Sci.*, vol. 12, no. 23, 2022, Art. no. 11992.

[6] G. Kang, J. Liu, Y. Xiao, B. Cao, Y. Xu, and M. Cao, "Neural and attentional factorization machine-based web API recommendation for mashup development," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4183–4196, Dec. 2021.

[7] C. Platzer and S. Dustdar, "A vector space search engine for Web services," in *Proc. 3rd Eur. Conf. Web Services*, 2005, pp. 1–9.

[8] J. Shen, W. Huang, and Q. Hu, "PICF-LDA: A topic enhanced LDA with probability incremental correction factor for Web API service clustering," *J. Cloud Comput.*, vol. 11, no. 1, p. 19, 2022.

[9] M. Shi, y. tang, and J. Liu, "Functional and contextual attention-based LSTM for service recommendation in mashup creation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 5, pp. 1077–1090, May 2019.

[10] J. Qin and Y. Jiang, "Recommender resources based on acquiring user's requirement and exploring user's preference with Word2Vec model in Web service," *Int. J. Internet Protocol Technol.*, vol. 12, no. 3, pp. 144–152, 2019.

[11] G. Kang, L. Ding, J. Liu, B. Cao, and Y. Xu, "Web API recommendation based on self-attentional neural factorization machines with domain interactions," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3953–3963, Nov./Dec. 2023.

[12] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist., Human Lang. Technol.*, 2019, pp. 4171–4186.

[13] Y. Zhong, Y. Fan, W. Tan, and J. Zhang, "Web service recommendation with reconstructed profile from mashup descriptions," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 468–478, Apr. 2018.

[14] S. Wang, Y. Zhou, and Z. Ding, "Automated restful API service discovery with various interface features," in *Proc. Int. Conf. Service-Oriented Comput.*, 2022, pp. 54–70.

[15] H. Mezni, S. Ait Arab, D. Benslimane, and K. Benouaret, "An evolutionary clustering approach based on temporal aspects for context-aware service recommendation," *J. Ambient Intell. Humaniz. Comput.*, vol. 11, no. 1, pp. 119–138, 2020.

[16] A. Pilani, K. Mathur, H. Agrawald, D. Chandola, V. A. Tikkiwal, and A. Kumar, "Contextual bandit approach-based recommendation system for personalized Web-based services," *Appl. Artif. Intell.*, vol. 35, no. 7, pp. 489–504, 2021.

[17] M. Shi, Y. Tang, and J. Liu, "TA-BLSTM: Tag attention-based bidirectional long short-term memory for service recommendation in mashup creation," in *Proc. Int. Joint Conf. Neural Netw.*, 2019, pp. 1–8.

[18] G. Kang et al., "TATT-BiLSTM: Web service classification with topical attention-based BiLSTM," *Concurr. Comput., Pract. Exp.*, vol. 33, no. 16, 2021, Art. no. e6287.

[19] G. Kang, J. Liu, Y. Xiao, Y. Cao, B. Cao, and M. Shi, "Web services clustering via exploring unified content and structural semantic representation," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4082–4096, Dec. 2022.

[20] Q. He et al., "Keyword search for building service-based systems," *IEEE Trans. Softw. Eng.*, vol. 43, no. 7, pp. 658–674, Jul. 2017.

[21] S. Lian and M. Tang, "API recommendation for Mashup creation based on neural graph collaborative filtering," *Connect. Sci.*, vol. 34, no. 1, pp. 124–138, 2022.

[22] R. Pan, Q. Yu, H. Xiong, and Z. Liu, "Collaborative recommendation algorithm based on deep graph neural network," *J. Comput. Appl.*, vol. 43, no. 9, p. 2741, 2023.

[23] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "LightGCN: Simplifying and powering graph convolution network for recommendation," in *Proc. Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2020, pp. 639–648.

[24] F. Xie, Y. Xu, A. Zheng, L. Chen, and Z. Zheng, "Service recommendation through graph attention network in heterogeneous information networks," *Int. J. Comput. Sci. Eng.*, vol. 25, no. 6, pp. 643–656, 2022.

[25] B. Jiang, H. Li, J. Yang, Y. Qin, L. Wang, and W. Pan, "Web service recommendation based on word embedding and node embedding," *Mobile Inf. Syst.*, vol. 2022, no. 1, pp. 1–12, Jul. 2022.

[26] X. Li, X. Zhang, P. Wang, and Z. Cao, "Web services recommendation based on metapath-guided graph attention network," *J. Supercomput.*, vol. 78, no. 10, pp. 12621–12647, 2022.

[27] Z. Zhang, Y. Zhang, M. Dong, K. Ota, Y. Zhang, and Y. Ren, "Collaborative tag-aware graph neural network for long-tail service recommendation," *IEEE Trans. Services Comput.*, vol. 17, no. 5, pp. 2124–2138, Sep./Oct. 2024.

[28] T. Yu, D. Yu, D. Wang, and X. Hu, "Web service recommendation for mashup creation based on graph network," *J. Supercomput.*, vol. 79, no. 8, pp. 8993–9020, 2023.

[29] N. Agarwal, G. Sikka, and L. K. Awasthi, "A systematic literature review on Web service clustering approaches to enhance service discovery, selection and recommendation," *Comput. Sci. Rev.*, vol. 45, Aug. 2022, Art. no. 100498.

[30] H. Wu, Y. Duan, K. Yue, and L. Zhang, "Mashup-oriented Web API recommendation via multi-model fusion and multi-task learning," *IEEE Trans. Services Comput.*, vol. 15, no. 6, pp. 3330–3343, Nov./Dec. 2022.

[31] T. Yu, L. Zhang, H. Liu, H. Liu, and J. Wang, "Service recommendation based on contrastive learning and multi-task learning," *Comput. Commun.*, vol. 213, pp. 285–295, Jan. 2024.

[32] X. Wang, M. Xi, and J. Yin, "Functional and structural fusion based Web API recommendations in heterogeneous networks," in *Proc. IEEE Int. Conf. Web Services*, 2023, pp. 91–96.

[33] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[34] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," in *Proc. 25th Conf. Uncertain. Artif. Intell.*, 2012, pp. 452–461.

[35] P. H. Le-Khac, G. Healy, and A. F. Smeaton, "Contrastive representation learning: A framework and review," *IEEE Access*, vol. 8, pp. 193907–193934, 2020.

[36] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, "Momentum contrast for unsupervised visual representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 9729–9738.

[37] S. Rendle, "Factorization machines," in *Proc. IEEE Int. Conf. Data Min.*, 2010, pp. 995–1000.

[38] R. Ye et al., "Dynamic graph construction for improving diversity of recommendation," in *Proc. ACM Conf. Recomm. Syst.*, 2021, pp. 651–655.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.

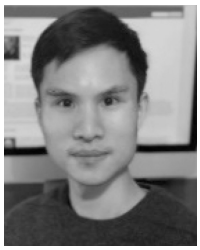
Guosheng Kang (Member, IEEE) received the Ph.D. degree in computer science from Fudan University, Shanghai, China. He is currently a Lecturer with the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan, China, and also with the Yunnan Key Laboratory of Service Computing, Yunnan University of Finance and Economics, Kunming, China. He has published more than 40 papers. His publications have appeared in some well-known conference proceedings and international journals, such as IEEE ICWS, IEEE SCC, ICSOC, IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING, *Future Generation Computer Systems*, and *Concurrency and Computation: Practice and Experience*. His research interests include service computing and business process management.

Hongshuai Ren received the B.S. degree in computer science from Yantai University, Yantai, China, in 2023. He is currently pursuing the M.S. degree in software engineering with the Hunan University of Science and Technology. He has published several papers in some well-known journals and conferences, such as IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT and ICSS. His current research interests include data mining and service computing.

Wanjun Chen is currently pursuing the B.S. degree in bigdata with the School of Computer Science and Engineering, Hunan University of Science and Technology. She has published several papers in some well-known conferences, such as CSCWD and ISPA. Her current research interests mainly include service computing and big data.

Jianxun Liu received the M.S. degree in computer science from the Central South University of Technology in 1997, and the Ph.D. degree in computer science from Shanghai Jiao Tong University in 2003. He is currently a Professor and the Dean of the School of Computer Science and Engineering, Hunan University of Science and Technology. He has published more than 80 papers in well-known conferences and journals, such as IEEE ICWS, IEEE SCC, *Future Generation Computer Systems*, and *Concurrency and Computation: Practice and Experience*. His current interests include workflow management, service computing, and cloud computing. He is the recipient of the second grade prize of both the Shanghai and Hunan Science Progress Award. He served as a Program Committee Member or the Program Chair of the major conferences in his area (IEEE ICWS, IEEE SCC, SCA, CGC, TrustCom, and DASC), and a Reviewer for IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, IEEE TRANSACTIONS ON SERVICES COMPUTING, and IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS.

Buqing Cao received the M.S. degree in computer science from the Central South University of Technology in 2007, and the Ph.D. degree from Wuhan University in 2011. He is currently a Professor and the Vice Dean of the School of Computer Science and Engineering, Hunan University of Science and Technology. His current interests include service computing, social network, and software engineering. He has published more than 50 papers in well-known conferences and journals, such as IEEE ICWS, IEEE SCC, JWSR, IEEE BigData, *China Communications*, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TIST, and IEEE TRANSACTIONS ON ARTIFICIAL INTELLIGENCE.



Yu Xu (Member, IEEE) received the Ph.D. degree in intelligence systems from Trinity College Dublin, Dublin, Ireland. He is currently a Postdoctoral Fellow with the Cambridge Baker Systems Genomics Initiative and the British Heart Foundation Cardiovascular Epidemiology Unit, Department of Public Health and Primary Care, University of Cambridge, Cambridge, U.K. His current research interests include genotype/phenotype imputations and polygenic risk score predictions. He has published more than 20 papers in well-known conferences and journals, such as *Nature*, *Nature Genetics*, *Cell Genomics*, *Statistical Methods and Applications*, UMAP, and ECIR.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 06:35:04 UTC from IEEE Xplore. Restrictions apply.